KØBENHAVNS UNIVERSITET

Salary prediction based on text analysis of job postings from Indeed

Introduction to Social Data Science Summer 2021

This paper contains   characters

12. august 2026

Indhold

1 Introduction

In the field of Social Data Science, there is a great quantity of research based around job postings from online job portals. These job portals play an important role in an economy's employment, especially in terms of frictional unemployment (workers searching for new jobs or are transitioning from one job to another) and private companies seeking specific skills and experiences. Job posts allows for both companies and job seekers to find matches between one another, based on qualifications, location, salaries and interests. In result, this creates data, which can be collected from the websites and is of substantial worth for both researchers and private companies.

Online job posts, from job portals such as Indeed, contains extensive amounts of unstructured data, in form of job descriptions, job titles, pay range, location etc. All of this unstructured data can be manipulated and used to create big useful databases, which can aid in the analysis and prediction of salaries for job posts that does not provide an estimated salary.

The aim of this paper is to extract and analyze data based on job postings from Indeed, and try to predict salaries, with the use of tools such as web scrapping, data structuring and cleaning, natural language processing (NLP), text analysis, machine learning and regression models. In our analysis we utilize a logistic regression, a Lasso model and a Random Forest model. We find that all models are good prediction models based on the data, but can be subject to overfitting. Even after extensive tuning of the Lasso and Random Forest model it cannot be concluded that the data is fitted appropriately without overfitting. Although, the analysis can be used by job seekers to get an idea of what fields to specialize and educate themselves within, to receive the highest paying job possible, based on data from job posts - but bearing the potential bias in mind.

The project continues as the following. Section 2 covers a short literature review discussing prior studies of the topic. Section 3 introduces the models used in our project. Section 4 presents the data and how the data is preprocessed to be used for our models and a short exploratory section. Section 5 introduces the analysis. This section is divided in to two parts, namely regression and Random Forest results based on the prediction of above or below median salary, and the textual analysis which uses Bernoulli Naive Bayes method to predict keywords that are associated with high paying salaries. This only includes salaries which are above the 75th percentile. Section 6 discusses the limitation of our analysis and other potential ways to treat the data. Section 7 summarises our findings in the previous sections with a conclusion.

2 Literature Review

The process of scraping job portals and predicting salaries based on the data collected, is done numerous times in many social data science research. Jackman and Reid (2013) used machine learning to compare various regression methods to predict salary based on job title and job description, and then ranks which methods turned out to perform the best. Their findings were that random forest outperforms neural network, which outperforms lasso regression. Furthermore, they were able to predict the salary of a job using only the text description, within a mean absolute error (MAE) of £5000, claiming that this model would do a better job, predicting salary, than a human person would, and therefor concludes their study a success.

Martín, Mariello, Battiti and Hernández (2018), uses web scraping to collect their data, and machine learning to train and analyze their scraped data, and finds in their more advanced analysis of salary prediction, that experience tends to be more rewarding in terms of salary than education, and develops an "accurate salary-range classifier...". They setup a web scrapper, that scrapes continuously over a year, and deletes duplicates, and thereby gets a more representative result, whereas scrapping a job portal once only gets a still-picture of the job market.

Furthermore, Kaggle has hosted competitions providing data sets of the UK job market to predict salaries. The goal has been to predict salaries from job posts, based on title and description. There are several articles regarding how programmers and researchers has gone about this topic, all of which use different methods and models to obtain different results.

3 Models

In this section an introduction to the theories of logistic regression, random forest and cross validation techniques used for the analysis will be introduced. As the purpose of this paper is to predict which keywords in job titles and job description gives the highest salary, we use different approaches and multiple models to obtain the best predictor. However, it should be noted that every model has its pros and cons, and that different models have different weaknesses compared to others. The performance of each model is also highly dependent on the data used. The sklearn module in python are used for all our predictions and model training.

3.1 Logistic regression model and regularization

Logistic regression is a statistical model that can be used for binary classification or multi-class classification. We are making use of the binary classification as the analysis is to predict certain inputs of keywords, that gives a salary above the median.
The logistic regression uses a *logistic* or *logit* function which is equal to the natural logarithm of the *odds ratio.*

$$logit(p) = log\frac{p}{(1-p)} \tag{3.1}$$

The *odds ratio* is written $\frac{p}{(1-p)}$, where p stands for the probability of the event that we want to predict. The *logit* function takes as input values in the range from 0 to 1 and transforms them to values over the entire real-number range, which can be used to express a linear relationship between feature values and the log-odds[1]:

$$logit(p(y=1|x)) = w_0x_0 + w_1x_1 + \cdots + w_nx_n = \sum_{i=0}^{n} w_ix_i = \mathbf{w^Tx} \tag{3.2}$$

In (3.2), $p(y = 1|x)$ refers to the conditional probability that a particular sample belongs to class 1 given its features x, w is the associated weight of the features. When we are predicting the probability that a certain sample belongs to a particular class we are implicitly using the inverse form of the *logit*, which is a *sigmoid function*[2].

[1]Python Machine Learning, 2nd ed. (2017) by Sebastian Raschka & Vahid Mirjalili
[2]Python Machine Learning, 2nd ed. (2017) by Sebastian Raschka & Vahid Mirjalili

3.1.1 LASSO

Regularization is one way to tackle the problem of overfitting, which happens when a model learns the detail and noise in the training data such that it has a negative impact on the model performance. This approach adds additional information, and thereby shrinking the parameter values of the model to induce a penalty against complexity. One popular approach to regularized linear regression are the so-called Least Absolute Shrinkage and Selection Operator (LASSO). LASSO regression is a penalized model where we simply add the squared sum of the weights to our least-squares cost function such that the minimization problem is

$$J(w)_{LASSO} = \sum_{i=1}^{n} \left(y^{(i)} - \hat{y}^{(i)}\right)^2 + \lambda \sum_{j=1}^{k} |w_j| \tag{3.4}$$

λ, is a hyper parameter, which helps in optimizing the process of minimizing the model's cost function. Also, the LASSO method overcomes the overfitting problem by punishing irrelevant weights, w_j, by setting them to 0.

3.2 Random Forest

A random forest is a supervised machine learning algorithm that combines multiple decision trees. This technique is used to solve classification and regression problems. The Random Forest model consists of a large number of decision trees. A decision tree consists of three components: a root node, a decision node and a leaf node. The decision tree is iterative splitting a training data set into branches from its nodes until nodes of leaves are attained[3]. The Random Forest model is a sensitive model where the objective is to tune a model that builds a large number of uncorrelated trees in the forest. The parameters used to tune are in our model only the size of the trees in the forest (n-estimators in the Sklearn module in python). In short, the Random Forest model an ensembling learning model that uses multiple random decision trees for better model accuracy. It reduces the variance of the individual decision tree by randomly selecting trees and averaging their class prediction. The model is widely used and has a simple algorithm to it utilized by the Sklearn module in python. We use the model as it comes with high predictive accuracy, works well with missing data, and prediction are based on input features which is important in a classification scenario.

[3]Python Machine Learning, 2nd ed. (2017) by Sebastian Raschka & Vahid Mirjalili

3.3 Cross validation

Cross validation is a technique assessing out of sample performance and accuracy of the model. One simple form of cross validation is to split the data into train and test data randomly. This approach can cause instability because the model is trained on different combinations of data points each time we train it, this in turn, can give us different results every time. Another, is to create partitions of the data k times also known as K-fold cross validation. In our analysis we split the data into $k = 10$ different folds. The models is trained on $k - 1$ folds and the kth is tested for effectiveness of the model. This suggest that our model is trained 10 times, each time, 9 folds would be included in the training group while the 10th would be the testing group. This process is repeated until every k folds has served as a test set. The average of our k estimated accuracy is our cross validation accuracy. Cross validation k folds ensures that every observation in the original data set has the chance to be trained and tested and generally result in a model skill estimate with low bias and modest variance.

3.4 Bernoulli Naive Bayes

Bernoulli Naive Bayes is a classification algorithm based on machine learning and Bayes theorem, and is used to give the likelihood of occurrence of an event. It uses binary variables, and is based on the two criticised assumptions: (1) that all attributes are independent of each other, (2) and that all attributes are of equal importance. Bernoulli Naive Bayes is used to analyze and predict the highest paying jobs, based on keywords in the job description and title, i.e. how likely it is that the job is of high salary, given a keyword appears in the description.

4 Data

All data has been web scraped from Indeed.com which is the largest open source job portal in the world. By using Beautiful Soup module in python we scrape our data set and by using natural language processing and python data manipulation we process our data ensuring it is applicable for modelling and predictions. The intial web scraped data set is composed of 1419 individual job postings for states in United States before cleaning. This section will cover the ethical aspect of web scraping Indeed as well as the data cleaning process and exploratory analysis of the data.

4.1 Data ethics

Usage of websites data is a sensitive topic, since websites have various different terms of service and rules regarding their data. Collecting this data, and especially gaining access to it through web scraping, comes with a lot of ethical questions to address. To circumvent ethical issues related to data we took into considerations the principles regarding data ethics by Salganik (2017)[4]. The first out of four principles in Salganik's guidance when facing ethical uncertainty is *Respect for Persons*. This principle states that the researcher should treat all people as autonomous, meaning, the researcher should receive informed consent from participants in the form of an agreement. As to the data chosen for this paper, we see no violation of this principle since job ads does not contain any sensitive information about individuals. Secondly, *Beneficence* is a principle of balancing the risk and benefit profile of the research. In short, maximizing benefits and minimizing possible harms. In relation to this analysis, we see no violation of this principle either, since this paper is only intended for research and not to be published. In addition, we see no harm in predicting salary based on job ads, though it can help the job seeker to expect some salary certainty upon job match. The third principle, *Justice* addresses the distribution of the burdens and benefits of the research. This means, it should not be the case, that one group gets all the benefits while others bear the costs of the research. In our case, we do not intend to conduct any form of actions to undermine the businesses of Indeed, thus we are in compliance with the principle of justice. Lastly, the *Respect for Law and Public interest* principle has two components, that is, compliance and transparency-based accountability. Compliance stresses the researcher to identify relevant laws and acts accordingly to the term of service, while transparency-based urges the researcher to be clear about goals, hence take responsibility for their actions. We identified that Indeeds terms of service included a statement: "Use of any automated system or software to extract data from the Site is prohibited", which we considered not to be in violation of, based on the fact that we only performed short, manually activated, web scraping code, and did not "automate" any process to collect data over time. Additionally, we added a sleep function, to

[4]Matthew J. Salganik, *Bit by Bit: Social Research in the Digital Age*, (2017)

avoid causing any heavy requests to the website, added a header to inform Indeed of our intent, as well as our contact information in case they deemed our request to be violating their terms of service. After every scrape we saved the data such that we only needed to scrape if new data was necessary - resulting in maximum 1 scrape a day. Therefore, we argue that no principal or terms of service was violated in our project.

4.2 Web scraping process

Indeed daily posts more than 500.000 job ads on their site around the world which for web scraping purposes results in many pages. However, for simplicity we selected to only focus on fields within the spectrum of economics in the United States. Additionally, the job postings would have to specify a range of salary. Each page consists of 15 job ads and the number of pages changed daily for which our range would have to be edited accordingly.

Using Beautiful Soup, looping through HTML tree and iterating through all pages in the specified field we collected the following for each job posting:

- Job title
- Job summary
- Location
- Salary
- Company

Unfortunately, the entire job description is not available from the HTML included in a given indeed job posting and due to heavy computational restrictions we were only able to capture the first 150 words of the job summary. All of the above leaving us with five lists from which an initial data set was created consisting of **1419** of observations ready for cleaning.

4.3 Data cleaning process

Examining the initial data set 4/5 columns where text data which had to be boiled down to only consist of keywords valuable for modelling and prediction. To analyse the text data the in-build packages from NLTK where used, as well as manual removal of certain strings. The salary data was scraped as a string range of salary in either yearly, monthly or hourly pay (i.e. $50,000 - $60,000 a year). Multiple string operations had to be completed since the text in the data was often unique. After removing the dollar sign, other string variables included in the range (these could include "From, Up to"etc) and wage information string (a year, a month, an hour). To align and scale the salary properly the average of the range replaced the range as value of the salary. The monthly and hourly wage was scaled by 12 and 1801[5], respectively, to proxy as yearly wages. After calculating yearly wages a new binary column was created to map above(1) or below(0) median salary.

For further analysis of the data we created a new column called "Summary"which describes the job postings in keywords after which we adapted NLTK package to visualize the frequency of keywords. This procedure was completed for the entire data set. This data was also used to create a separate binary data set that will be used for LASSO and Random Forest regressions to predict salary given keywords in job summary.

However, not all job postings in the data set contained information about the salary and where useless for training our model. These observations where stored in a separate data set before training and testing our model leaving only 359 observation for test and training.

4.4 Data for textual analysis

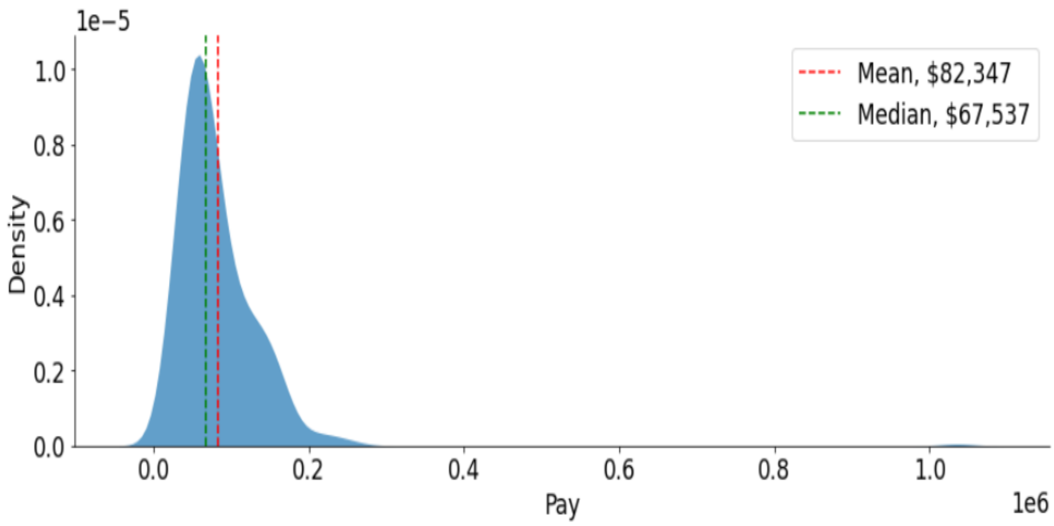
In addition to the prediction modelling above we added a textual analysis where the focus was to interpret and predict keywords that would be associated with job posting salary above the 75th percentile using a different approach, namely, Bernoulli Naive Bayes Theory. To classify high pay, we set up a boolean variable called "Pay high", which indicated whether the pay was in the top 75th percentile (above = 1). We used the upper 25th percentile of our own data, since economic related jobs were already

[5]A quick google search told us the average American works 37,5 hours pr week or 1801 hours a year

considered high salary, so we found it appropriate to not use a measurement based upon salaries in general. We split up our data set into test- and train data set and trained our train data set using the ”sklearn”machine learning library, which uses Bernoulli Naive Bayes theory to find the best possible prediction for which word correlates most with “Pay high” equaling 1.

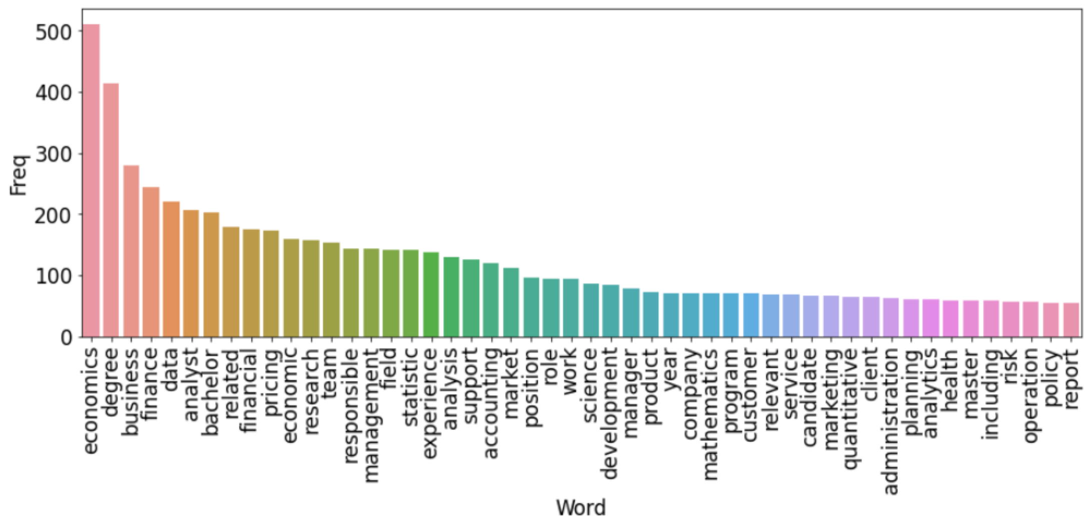
4.5 Exploratory data analysis

The distribution of salary informed in the job postings are plotted. The figure shows that the salary range does not differ so much from the median, although suggesting that the majority of economic job postings in this data set will be offered a slightly lower than median salary. The distribution plot also shows one outlier from a job posting offering a salary of $1 million that can be seen to the far right, perhaps skewing the distribution a little. Although, we thought it not of being too outrageous to remove it as an outlier. It is not unrealistic that a senior position in the United States would offer this yearly. Additionally, as the purpose of this assignment is to predict whether a salary will be above or below a median based on text data, the outlier wont affect any predictions.



Figur 1: Distribution of Indeeds salary range in job summaries

For further visualisation the top 50 words used in the job description for economics and the keywords used for mapping the train/test data set is depicted below. Where, quite obviously, economics is the most frequent.



Figur 2: Frequency of keywords in job description

As explained in our data section, the words mapped in the above is our data set for our prediction models. Additionally, words in job titles will be used where the most common words are in the same area with "analyst, pricing, research" etc. yielding nice information that economists more than often are wanted in advanced positions such as the above[6].

5 Results

In this section the results from our textual analysis, the three model performance metrics for LASSO and Logistic regression in terms of root- and mean squared error as well as the accuracy from the Random Forest model will be presented.

5.1 Logistic, Lasso and Random Forest performance

To evaluate the salary prediction obtained from the models the performance metrics are presented in table 1 and 2. The measures include the mean squared error and the root mean squared error for modelling job title and summary description, respectively.

For the model to yield reliable performances to be trusted and used as a predictor it is important that the errors are minimized. Table 1 shows that the logistic regression

[6]Graphic of top 50 words in job titles are depicted in appendix A.

Tabel 1: Performance metric results from Logistic and LASSO regression

	Title		Summary	
	MSE	RMSE	MSE	RMSE
Logistic Regression	0.328	0.572	0.328	0.61
LASSO	0.229	0.485	0.227	0.51

has larger values of root- and mean squared errors suggesting that the model is worse at predicting the salary based on the keywords in our two data sets. As the Lasso is a extended version of the logistic regression penalising overfitting we try to utilise the lambdas in the Lasso regression to find the best penalty variable to tune the best model for this data, for which graphs of convergence is presented in the Appendix. The effort resulting in a better model based on the performance metrics with a mean squared error of 0.22 for both Job Title and Summary.

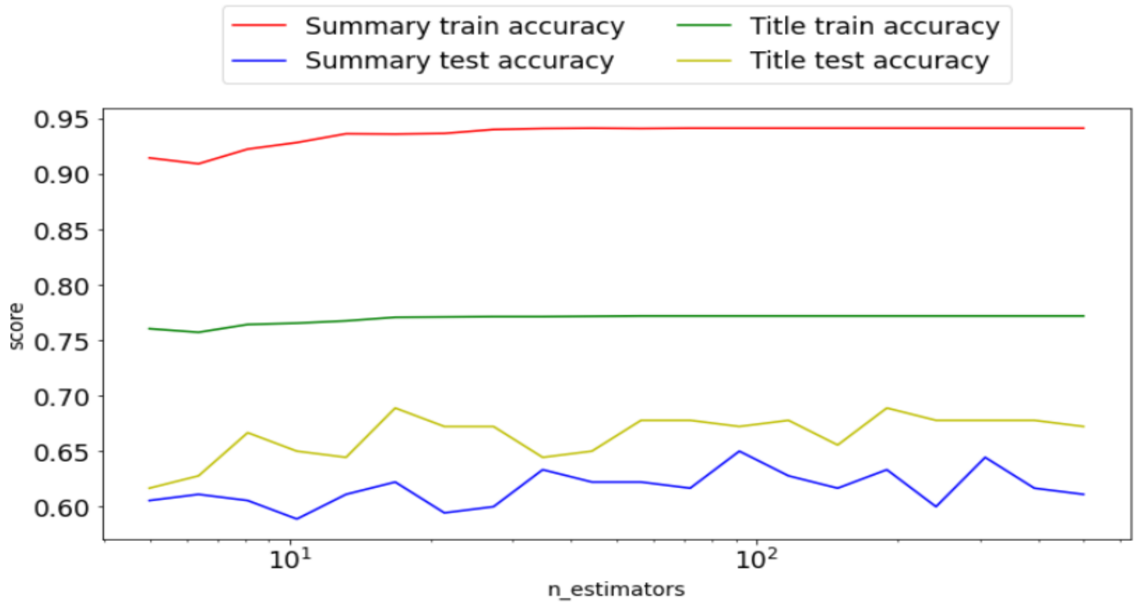
For the purpose of obtaining an even better prediction model we consider the Random Forest model. The model works by creating several decision trees during training time and output the mean of all classes as the prediction of all its trees. We trained the model by selecting n-estimators (no. of trees in the forest) to an array of values from 0 to 500 as the performance of a random forest classifier is highly sensitive to change of hyper parameters. Due to sensitivity we decided to only tune on n-estimators and reporting the maximum score from the train and test data. This is reported in table 2 below.

Tabel 2: Random Forest accuracy

	Train Score	Test Score
Title	0.771	0.7
Summary	0.941	0.65

The results suggests that the model has an prediction accuracy of 77% when the model is trained on the job titles and 94% when trained on keywords in the summary. Suggesting that the Random Forest is a more powerful model than the two above given this data. It yields very high prediction rates suggesting that based on this model a job seeking economist would be able to predict whether or not a job posting will be above median pay or below, allowing her to segment job ads when searching for an employer.

Depicting the training and test values from the Random Forest classifier for n-estimators, seen below, could however suggest the model in fact is overfitting. Especially when training one the summary data. This is discussed further in the model limitation section.



Figur 3: Random Forest accuracy for n$_e$*stimators*

5.2 Textual analysis

Table 3 shows our models prediction of highest salary-associated keywords. As we can see, "Senior", "Directors", "Manager" etc. are classified as high paying indicating keywords, which is also what we would expect, since these words often are included in job titles of a high position in firms. If we compare our models high paying keywords, to Indeeds own article about “Top Economics Degree Jobs”, we can see that eight out of ten high paying keyword in our model, also appears in a job description or title in Indeeds article. We interpret this as being a good indication that our model works, in terms of catching on to which keywords are associated with a high salaries. This means that if a job description includes one or more of these keywords, our model predicts that the job also comes with a high paying salary.

Tabel 3: Top salary associated keywords

High paying keywords
Investment
Directors
Manager
Senior
Officer
System
Analytics
Officer
Business
Consultant

6 Discussion and limitations of the project

In the following section the analysis will be discussed and reflected upon. Furthermore, limitations of the models will be discussed.

The process of web scraping and creating a data frame based on text data can be a heavy computational task. The job ads obtained from Indeed come in different writing styles and priorities for what the summary should include depends on the person and company posting the job. This can result in many different types textual data obtained. One example of this could be how our code mainly strips text as sections and when the job summary would include bullet points the code would not include the summary. Although removed from our data it leaves out many job summaries that could be included. Additionally, problems with text formatting resulted in some words not getting removed when cleaning the data. This task would thus be done manually which could end up with words not being removed properly. As consequence these words would be included when stemming and tokenizing our data that could influence the modelling. This was especially the case for our job titles data where abbreviations such as "FPA, sr, u, ap"where not removed. Investigating the top words the abbreviations did not appear before the 27th top word being "sr"which can be argued should include as it captured a level of seniority.

Having specified a specific job area with only one word "economics" it is clear that all job titles will have economics involved. Meaning that economics will be included equally in high paying jobs and low paying jobs, suggesting that the word economics could be seen as a futile measure. Nonetheless we keep it when modelling as we suspect the word will draw the prediction in both directions and thus not be biased towards low paying or high paying jobs.

We use different regression models and a Random Forest model to try to avoid overfitting of the model. Firstly, it is clear for us that without any type of regularization or tuning of the models would result in overfitting. This was the case for the standard logistic regression. Hence introducing the Lasso regression, here crucial to optimize the lambda regurlariser we attempted to see how the convergence of the data looked by iterating through a large spectrum of lambdas. Show in appendix. However, the optimal lambda was not used in modelling, but rather the average of all the lambdas. Hence, not being entirely sure if the data overfits with this model. This lead to using a Random Forest classifier. This model performs very well in prediction of the data, perhaps worryingly well. With a prediction rate of 77% in the train data and 94% for test data is could be argued that the data in this case also is overfitting.

This suggests that further validation measures of the models could be necessary. If the time allowed it, another interesting analysis would be to investigate if we could predict an actual salary value given the job title or job summary, rather than limiting for below/above median and high or low paying jobs. Additionally, with more practise in web scraping it would be interesting to scrape more data from indeed such as skills and company reviews. Lastly, for further analysis the job postings data could be used differently. Such as predicting the economic situation in the United Stated based on no. of job posting - or companies stock price compared with number of job openings.

Our machine learning process for Bernoulli Naive Bayes has an obvious flaw in that it cannot separate which words are very common in job descriptions in general, and therefore registers them as both high and low pay, since they appear a lot in general. In our project, our model caught the word "analyst", as both a high and a low paying keyword. Further analysis showed that "analyst" was a very frequent word in our data set, which makes sense as to why this error occurs. This could be fixed through more advanced coding, but for our project we have chosen to ignore it in our data, and taking it into account when analyzing our result. We have also chosen to ignore top "high paying keywords", that is considered "stop words", since they are not of interest, and it is an error that our cleaning of stop words did not catch these.

With a bigger data set we would be able to see a more accurate prediction, of which words were associated with a high pay, as our machine learning model, would have more input to find correlation between these words and predicted salary. However, creating a model like this using social data science, we don't expect it to work flawlessly without errors. The output should always be carefully interpreted, since there can be a lot of reasons as to why a keyword stands out to be a high paying keyword indicator, and it is highly dependent on the data we use.

7 Conclusion

This paper aimed at predicting salaries based on job ads from Indeed, thus we were using Beautiful soup to scrape the data and ended up with 1419 observations within the spectrum of economics. The process of cleaning the data for predictive usage, involved, manual removal of certain strings and using the in build packages from NLTK, scaling etc. leaving us with 369 observations. The analysis showed that using a Random Forest model one would achieve a prediction accuracy of 77% when training on Job Titles and 94% when the model is trained on job summary. However, we cannot conclude that the model is not overfitting the data - which has to be in mind if used as a segmentation tool. Generally for all our models in section 5.1. it is unclear if the data is in fact fitted properly. Our textual analysis of top high paying keywords, included a lot of seniority position job title words, as well as overlapping with Indeeds own top 18 paying job posts, which indicates our model was able to capture high paying keywords through Bernoulli Naive Bayes theory.

Litteratur

[1] Shaun Jackman and Graham Reid, Predicting Job Salaries from Text Descriptions, University of British Columbia, 2013-04-16

[2] Ignacio Martin, Andrea Mariello, Roberto Battiti, José Alberto Hernández, Salary Prediction in the IT Job market with Few High-Dimensional Samples: A Spanish Case Study, Atlantis Press, July 2018

[3] Indeed Editorial Team, ”18 Top Economics Degree Jobs”, June 10, 2021

[4] Python Machine Learning, 2nd ed. (2017) by Sebastian Raschka & Vahid Mirjalili

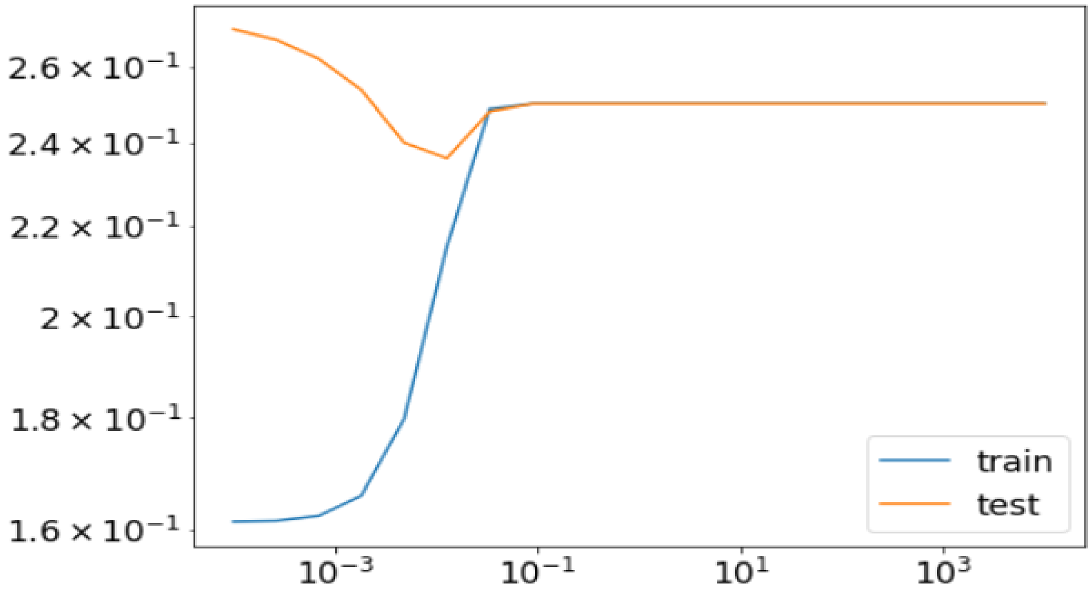
[5] Matthew J. Salganik, Bit by Bit: Social Research in the Digital Age, (2017)

[6] Michael Salmon, Medium.com June 2017

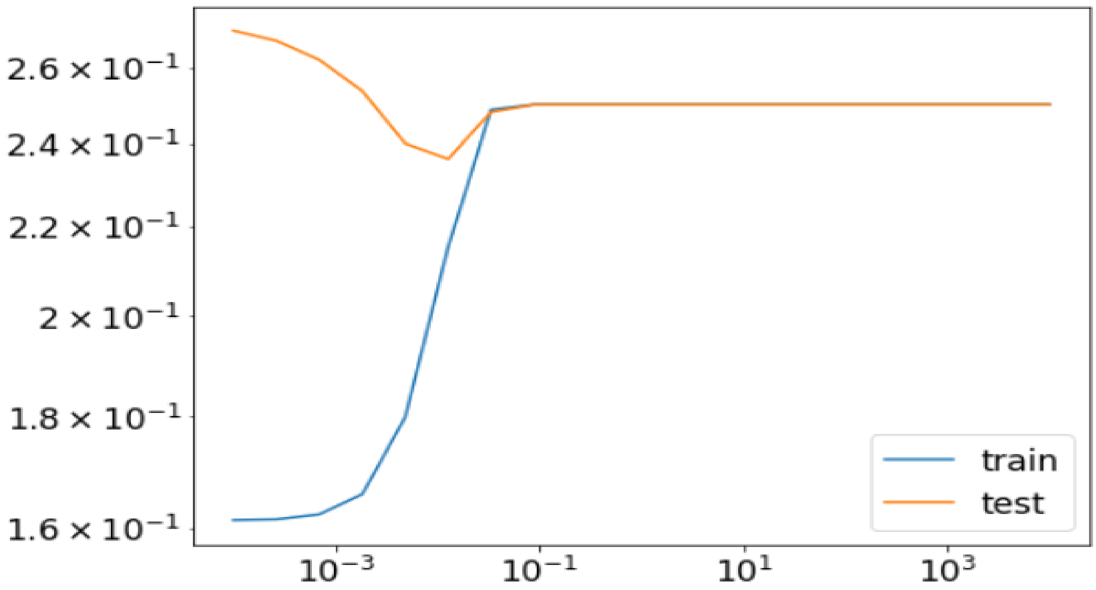
[7] Dutta et al, working paper ”Design of a novel Prediction Engine for predicting suitable salary for a Job”(2018)

8 Appendix

8.1 Lasso regression convergence for increasing lambdas

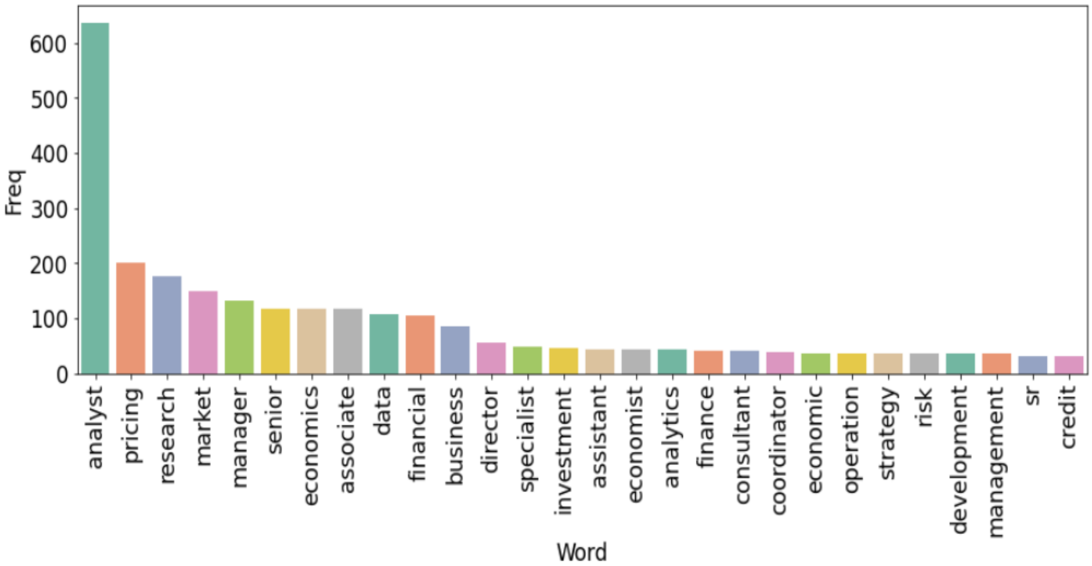


Figur 4: RMSE convergence for Job title



Figur 5: RMSE convergence for Job Summary

8.2 Job Title top 50 words



Figur 6: Word frequency for Job titles